

Public	OBJECTIF PÉDAGOGIQUE
Ingénieur 1 (Bac+3)	OBJECTIF PÉDAGOGIQUE : Maîtriser les stratégies et outils de test logiciel pour garantir la qualité d'une application tout au long de son cycle de développement. Être capable de mettre en place une suite de tests complète (unitaires, intégration, E2E) et un pipeline de déploiement continu sécurisé en environnement de production.
Durée	
3 demi-journées	
Prérequis	
Dev B1/B2 DevOps ING1 Git maîtrisé	Objectifs Opérationnels : À la fin du module, les apprenants seront capables de : - Comprendre la pyramide des tests et les différents niveaux - Écrire des tests unitaires efficaces avec Jest ou Pytest - Mettre en place des tests d'intégration et de contrats d'API - Automatiser les tests end-to-end avec Cypress ou Playwright - Mesurer et améliorer la couverture de code - Intégrer les tests dans un pipeline CI/CD - Déployer une application en production avec rollback automatique
Modalités d'évaluation	
TP suite de tests + déploiement noté	
Modalités pédagogiques	
Cours TP sur machine	PROGRAMME
Matériel et logiciel	STRATÉGIE DE TEST - Pyramide des tests : unitaires, intégration, E2E - TDD : Red-Green-Refactor cycle - BDD : Gherkin, Given-When-Then, Cucumber (notions) - Shift-left testing : tester tôt pour réduire les coûts - Critères de qualité : FIRST (Fast, Isolated, Repeatable, Self-validating, Timely) TESTS UNITAIRES - Jest (JS/TS) : describe, test/it, expect, matchers - Pytest (Python) : fixtures, parametrize, assertions - Mocking : jest.mock, jest.fn, spies, stubs - Test des fonctions pures, des modules et des classes - Couverture de code : Istanbul/nyc, coverage report TESTS D'INTÉGRATION - Tests d'intégration vs tests unitaires : périmètre et objectifs - Test des routes API : Supertest (Node.js), httpx (Python) - Base de données de test : in-memory (SQLite), fixtures, seeds - Contract testing : Pact — vérification des contrats API - Tests de composants React : React Testing Library TESTS END-TO-END - Cypress : installation, configuration, structure des tests - Sélecteurs : cy.get, cy.contains, data-testid - Actions : click, type, submit, navigation - Assertions : should, expect - Playwright (notions) : avantages multi-navigateurs
Modalités particulières	
Classe Inversée	
Oui Non	

► Compétences (*)	PROGRAMME (SUITE)
Compétence : Garantir la qualité d'une application par les tests et le déploiement Critère : Suite de tests > 80% de couverture, CI/CD	TESTS DE PERFORMANCE ET DE SÉCURITÉ • Tests de charge : k6, Artillery — scénarios de montée en charge • Métriques de performance : temps de réponse, percentiles p95/p99 • Tests de sécurité SAST : SonarQube, ESLint security • OWASP Top 10 : tests des vulnérabilités courantes • Dependency scanning : Snyk, Dependabot INTÉGRATION DES TESTS DANS CI/CD • Parallélisation des tests dans GitHub Actions • Gating : bloquer le déploiement si les tests échouent • Rapports de tests : JUnit XML, GitHub Checks • Notifications : Slack, email sur échec de pipeline • Test dans plusieurs environnements : staging, pre-prod STRATÉGIES DE DÉPLOIEMENT • Blue/Green deployment : bascule sans interruption • Canary release : déploiement progressif sur un sous-ensemble • Feature flags : activation sélective de fonctionnalités • Rollback automatique : détection de régression et retour arrière • TP : déploiement Blue/Green sur un serveur Linux QUALITÉ ET DETTE TECHNIQUE • Code review : checklist, bonnes pratiques, outils • SonarQube : analyse statique, technical debt, code smells • Métriques de qualité : maintenabilité, fiabilité, sécurité • Refactoring sécurisé : technique des tests de caractérisation • Documentation vivante : tests comme spécifications PROJET FINAL ET ÉVALUATION • TP noté : suite de tests complète (unit + intégration + E2E) • Pipeline CI/CD avec déploiement automatique en staging • Rapport de couverture de code et analyse SonarQube • QCM de validation des acquis tests & déploiement
► Niveau visé	
Intermédiaire	
► Outils utilisés	
Jest / Pytest Cypress GitHub Actions	
► Contrôle des acquis	
TP tests + déploiement	